Universidad Nacional Autónoma de México
Facultad de Ingeniería

Bases de Datos

Grupo: 01

Tarea 08. Tipos de datos Postgresql

Profesor: FERNANDO ARREOLA FRANCO

Alumno: López Romero David Baruc
No. de Cuenta: 321018067
Semestre: 2026-2

Ciudad Universitaria, CDMX
Abril 2026

1. Tipos Numéricos

PostgreSQL proporciona una amplia gama de tipos numéricos. Los tipos enteros son los más comunes, mientras que los tipos de punto decimal se dividen en exactos e inexactos.

Cuadro 1: Principales tipos numéricos en PostgreSQL

Tipo	Almacenamiento	Rango	Descripción
integer	4 bytes	-2,147,483,648 a +2,147,483,647	Entero estándar.
numeric	Variable	Hasta 131,072 dígitos	Precisión exacta.
real	4 bytes	6 decimales de precisión	Punto flotante.
serial	4 bytes	1 a 2,147,483,647	Autoincrementable.

Ejemplo de implementación y uso:

```sql
-- Creacion de la tabla
CREATE TABLE finanzas (
    id serial PRIMARY KEY,
    monto numeric(10, 2),
    cantidad integer
);

-- Insercion de datos
INSERT INTO finanzas (monto, cantidad) VALUES (1250.50, 10);
```

2. Tipos de Carácter

Para estos tipos, la elección depende principalmente de si se desea imponer una restricción de longitud.

- **varchar(n):** Longitud variable con límite.
- **char(n):** Longitud fija, rellena con espacios.
- **text:** Longitud ilimitada (recomendado en Postgres).

Ejemplo de implementación y uso:

```sql
CREATE TABLE usuarios (
    username varchar(25) UNIQUE,
    perfil text
);

INSERT INTO usuarios (username, perfil)
VALUES ('d_baruc', 'Estudiante de Ingenieria interesado en bases de datos.');
```

3. Tipos de Fecha

El manejo de fechas es una de las mayores fortalezas de PostgreSQL, permitiendo cálculos precisos entre periodos.

Cuadro 2: Tipos de datos cronológicos

Tipo	Almacenamiento	Rango	Formato
date	4 bytes	4713 BC a 5874897 AD	YYYY-MM-DD
timestamp	8 bytes	4713 BC a 294276 AD	YYYY-MM-DD HH:MM:SS
interval	16 bytes	+/- 178,000,000 años	Cantidad de tiempo

Ejemplo de implementación y uso:

```
CREATE TABLE eventos (
    nombre_evento text,
    fecha_inicio timestamp,
    duracion interval
);

INSERT INTO eventos VALUES ('Examen BD', '2026-04-22 07:00:00', '2 hours');
```

4. Tipo JSONB (JSON Binario)

El acrónimo JSON significa *JavaScript Object Notation*. PostgreSQL ofrece dos tipos para manejar este formato: `json` y `jsonb`. La principal diferencia radica en su almacenamiento; mientras que `json` guarda una copia exacta del texto de entrada (incluyendo espacios en blanco), `jsonb` (JSON binario) descompone y almacena los datos en un formato binario personalizado.

¿Para qué se utiliza? Se utiliza cuando se necesita flexibilidad en el esquema de la base de datos (característica típica de bases NoSQL como MongoDB) pero sin perder los beneficios de una base de datos relacional (ACID). Es ideal para:

- Almacenar configuraciones o preferencias de usuario que varían mucho de un perfil a otro.
- Guardar respuestas dinámicas de APIs externas.
- Soportar esquemas de datos que cambian con demasiada frecuencia y donde hacer un `ALTER TABLE` constante no es viable.

La gran ventaja de `jsonb` es que soporta indexación (especialmente índices GIN), lo que hace que las búsquedas dentro de las llaves del JSON sean extremadamente rápidas.

Ejemplo de implementación y uso:

```sql
-- Creacion de la tabla para preferencias
CREATE TABLE configuraciones (
    usuario_id integer PRIMARY KEY,
    ajustes jsonb
);

-- Insercion de un documento JSON
INSERT INTO configuraciones (usuario_id, ajustes)
VALUES (1, '{"tema": "oscuro", "notificaciones": true, "idioma": "es"}');
```

5. Tipo UUID

El tipo de dato `uuid` (*Universally Unique Identifier*) almacena valores de 128 bits generados por un algoritmo diseñado para garantizar que el identificador sea único, no solo en una tabla o base de datos, sino en todo el mundo. Cumple con el estándar definido por el RFC 4122.

¿Para qué se utiliza? Tradicionalmente, las llaves primarias se hacen con números enteros autoincrementales (`serial`). Sin embargo, los UUIDs son preferidos en arquitecturas modernas por las siguientes razones:

- **Sistemas distribuidos:** Si tienes varias bases de datos o microservicios generando registros al mismo tiempo, usar un UUID evita colisiones de IDs al sincronizar los datos.
- **Seguridad:** Un ID autoincremental expone cuántos registros tienes (si mi ID es el 50, sé que hay 49 antes que yo). Un UUID es impredecible.
- **Operaciones *offline*:** Permite que una aplicación móvil genere un registro con su ID sin necesidad de conexión, y lo inserte en el servidor posteriormente sin conflictos.

Ejemplo de implementación y uso:

```sql
-- Creacion de tabla con UUID como llave primaria
CREATE TABLE dispositivos (
    device_id uuid PRIMARY KEY,
    modelo text,
    estado varchar(15)
);

-- Insercion de datos usando formato hexadecimal
INSERT INTO dispositivos (device_id, modelo, estado)
VALUES ('a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a11', 'ThinkPad X1', 'Activo');
```

6. Tipo Point (Geométrico)

PostgreSQL cuenta con soporte nativo para tipos de datos bidimensionales orientados a la geometría (como líneas, cajas, caminos y polígonos). El tipo `point` es el bloque fundamental de esta familia, representando un único punto en un plano cartesiano mediante un par de coordenadas (x, y).

¿Para qué se utiliza? Aunque para sistemas de información geográfica (GIS) complejos se suele instalar la extensión *PostGIS*, los tipos geométricos nativos son excelentes para aplicaciones más simples:

- Almacenar ubicaciones en mapas usando latitud y longitud.
- Calcular distancias directas entre dos puntos dentro de la misma consulta SQL mediante operadores nativos (como el operador `<->` para distancia).
- Dibujo industrial o sistemas de seguimiento en planos de dos dimensiones.

Ejemplo de implementación y uso:

```sql
-- Creacion de la tabla espacial
CREATE TABLE ubicaciones (
    nombre_lugar text,
    coordenadas point
);

-- Insercion de datos con coordenadas (Longitud, Latitud)
INSERT INTO ubicaciones (nombre_lugar, coordenadas)
VALUES ('Facultad de Ingenieria', point(-99.1872, 19.3321));
```

Referencias

[1] The PostgreSQL Global Development Group, *PostgreSQL 16.2 Documentation*, 2024. [Online]. Available: https://www.postgresql.org/docs/16/datatype.html